

Rogert

Osoba 2 - Prymitywy 3D, przeszkody i kolizje

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/PrimitiveNode.h, Engine/PrimitiveNode.cpp, Demo/Obstacles.cpp

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Rogert).

```
pgk/
|
|-- Engine/
|   |-- Math3D.* SceneNode.*          (Lukasz)
|   |-- PrimitiveNode.h    <<< MOJE  (wszystkie kształty 3D)
|   |-- PrimitiveNode.cpp <<< MOJE
|   |-- Camera.* Light.* Texture.*    (Kacper)
|   |-- Engine.*              (Jakub)
|
|-- Demo/
|   |-- Room.cpp              (Lukasz)
|   |-- Obstacles.cpp        <<< MOJE (kolumny, stożki, skrzynie + kolizje)
|   |-- Targets.cpp          (Kacper)
|   |-- Gameplay.cpp Constants.h ShootingGallery.h (Jakub)
|
|-- main.cpp CMakeLists.txt      (Jakub)
```

W skrócie: ja robię **kształty** (szescian, walec, sfera, stożek, torus, płaszczyzna) - każdy rysowany własnym kodem OpenGL - oraz **przeszkody** w grze wraz z **kolizjami** (gracz nie przechodzi przez ściany, strzał nie przelatuje przez kolumny).

1. O co chodzi w moich plikach (po ludzku)

Komputer nie wie, czym jest 'szescian'. Trzeba mu podać każdy **wierzchołek** (rog), **normalna** (kierunek, w który patrzy powierzchnia - potrzebny do światła) i **współrzędne UV** (gdzie na teksturze jest dany punkt). To właśnie robię w **PrimitiveNode** - ręcznie, przez glBegin/glEnd. Nie używam gotowców typu glutSolidCube, bo wymaganiem projektu było napisanie geometrii samemu.

W **Obstacles.cpp** używam tych kształtów, żeby postawić w pokoju kolumny, stożki i skrzynie. Dopusuje też **matematykę kolizji**: żeby gracz nie wchodził w przeszkody i żeby przeszkoda zasłaniała strzał.

2. PrimitiveNode - wspólna baza wszystkich kształtów

PrimitiveNode dziedziczy po **SceneNode** (Lukasz) i dodaje **material** (kolory dla światła) oraz **teksturę**. Ma jedną wspólną metodę **renderSelf**, która ustawia material/teksturę, a potem woła **drawGeometry()** - i to właśnie ta funkcja jest różna dla każdego kształtu.

PrimitiveNode.cpp - wspólny render, geometria w drawGeometry()

```
void PrimitiveNode::renderSelf(const Mat4& worldMatrix) const {
    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, ambient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, diffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, specular);
    glMaterialf (GL_FRONT_AND_BACK, GL_SHININESS, shininess);
    glColor3f(diffuse.x, diffuse.y, diffuse.z); // fallback gdy światło off

    bool hasTexture = (surfaceTexture && surfaceTexture->isLoaded());
    if (hasTexture) { glEnable(GL_TEXTURE_2D); surfaceTexture->bind(); }

    glPushMatrix();
    glMultMatrixf(worldMatrix.data()); // ustaw transformację obiektu
    drawGeometry(); // <- nadpisane w każdej podklasie
    glPopMatrix();

    if (hasTexture) { surfaceTexture->unbind(); glDisable(GL_TEXTURE_2D); }
}
```

drawGeometry() jest **czysto wirtualna** (= 0) - to znaczy, że **PrimitiveNode** sam jej nie ma, a każda podklasa MUSI ją napisać. To wzorec 'szablon metody': baza robi powtarzalne rzeczy (material, transformacja), a detale (jaki kształt) zostawia potomkom.

3. Jak rysuje konkretne kształty

3.1. Sfera - siatka równoleżników i południków

Sfere dzieli na poziome pasy (stacks) i pionowe wycinki (slices), jak globus. Każdy punkt liczy ze współrzędnych sferycznych. Ważne: dla sfery jednostkowej **normalna = pozycja punktu** (kierunek od środka).

SphereNode::drawGeometry() - jeden wierzchołek

```
float nx = sin(theta)*cos(phi);
float ny = cos(theta);
float nz = sin(theta)*sin(phi);
glNormal3f(nx, ny, nz); // normalna = kierunek na zewnątrz
glTexCoord2f(u, v); // u = phi/2pi, v = theta/pi
glVertex3f(nx*radius, ny*radius, nz*radius);
```

3.2. Stożek - dlaczego osobne trojkąty, a nie wachlarz

Stozek mozna narysowac jako TRIANGLE_FAN z jednym wierzchołkiem na czubku. Ale wtedy czubek ma **jedna** normalna, przez co cieniowanie wyglada plasko/brzydko. Dlatego rysuje N osobnych trojkatow - kazdy ma na czubku **usredniona** normalna z dwoch sasiednich scianek. Wyglada duzo lepiej.

ConeNode::drawGeometry() - nachylenie normalnej zalezy od proporcji r:h

```
float L = sqrt(r*r + h*h);
float nh = h / L;    // pozioma skladowa normalnej
float nv = r / L;    // pionowa skladowa (stozek zwęża sie ku gorze)
// normalna sciany bocznej w kierunku kata 'ang':
glNormal3f(cos(ang)*nh, nv, sin(ang)*nh);
```

3.3. Plaszczyzna i skala UV

PlaneNode to zwykly prostokat w plaszczyznie XZ z normalna w gore. Dodalem **setUVScale**, bo na duzej scianie tekstura powtarzala sie zbyt gesto.

PlaneNode::drawGeometry() - UV mnozone przez skale

```
// uMax/vMax decyduja ile razy tekstura sie powtorzy
const float uMax = planeWidth * planeUVScale;
const float vMax = planeDepth * planeUVScale;
glTexCoord2f(0.0f, 0.0f); glVertex3f(-hw, 0, hd);
glTexCoord2f(uMax, vMax); glVertex3f( hw, 0,-hd); // itd.
```

4. Demo/Obstacles.cpp - przeszkody i kolizje

Tu stawiam w pokoju: 6 kolumn (walce), 3 stożki i 2 skrzynie (szesciany). Każda przeszkoda zapisuje w dwóch miejscach: jako **wezel sceny** (zeby ja narysowac) i jako **dane kolizji** (zeby liczyć fizyke).

4.1. Trzy rodzaje kolizji gracza

Gracz ma 'hitbox' - kołko o promieniu PLAYER_RADIUS. Po ruchu sprawdzam, czy wszedł w przeszkode, i jak tak - **wypycham** go na zewnątrz.

Obstacles.cpp - wypychanie gracza z kolumny

```
// kolizja z walcem (kolizja kolowa w płaszczyźnie XZ)
float dx = playerEye_.x - o.base.x;
float dz = playerEye_.z - o.base.z;
float minR = o.radius + PLAYER_RADIUS;
float distSq = dx*dx + dz*dz;
if (distSq < minR*minR && distSq > 0.0001f) {
    float dist = sqrt(distSq);
    float push = (minR - dist) / dist; // ile brakuje do wyjścia
    playerEye_.x += dx * push;         // wypchnij wzdłuż osi środek->gracz
    playerEye_.z += dz * push;
}
```

Dla **skrzyni** (prostokąt) szukam najbliższego punktu na jej brzegu i wypycham gracza o promień. Dla **stożka** traktuje go jak wałek o promieniu podstawy (gracz i tak nie wejdzie pod nawis).

4.2. Przeszkoda zasłania strzał

Gdy gracz strzela, sprawdzam czy **najpierw** nie trafi w przeszkode. Używam funkcji raycast od Lukasa.

Obstacles.cpp - czy przeszkoda jest bliżej niż cel

```
bool ShootingGallery::obstacleBlocksRay(const Vec3& origin,
                                         const Vec3& dir, float maxT) const {
    float t;
    for (const auto& o : cylinderObs_)
        if (rayCylinderIntersect(origin, dir, o.base, o.radius, o.height, t)
            && t > 0.0001f && t < maxT) return true;
    for (const auto& o : coneObs_)
        if (rayConeIntersect(origin, dir, o.base, o.radius, o.height, t)
            && t > 0.0001f && t < maxT) return true;
    for (const auto& o : boxObs_)
        if (rayAABBIntersect(origin, dir, o.boxMin, o.boxMax, t)
            && t > 0.0001f && t < maxT) return true;
    return false;
}
```

maxT to odległość do trafionego celu. Jeżeli któraś przeszkoda jest bliżej ($t < \text{maxT}$), strzał jest zablokowany i liczy się jako pudło - cel był schowany za kolumną.

5. Pytania od wykładowcy - przygotowanie

PrimitiveNode - ogólne

P: Dlaczego nie używacie gotowych funkcji jak `glutSolidSphere`?

O: Bo wymaganiem projektu było, żeby grupa sama napisała geometrie prymitywów. FreeGLUT ma tylko pomagać z oknem i wejściem. Dlatego każdy kształt rysuje ręcznie przez `glBegin/glEnd`, samodzielnie licząc wierzchołki, normalne i UV.

P: Co to jest normalna i po co ją podajesz?

O: Normalna to wektor prostopadły do powierzchni - mówi, w którą stronę 'patrzy' ścianka. OpenGL używa jej do liczenia oświetlenia: im bardziej normalna skierowana do światła, tym jaśniejszy punkt. Bez normalnych model byłby płaski, bez cieniowania.

P: Co to są współrzędne UV?

O: To współrzędne na teksturze (od 0 do 1). Mówią, który fragment obrazka nalepić na dany wierzchołek. `glTexCoord2f(u,v)` przed `glVertex3f` przypisuje punktowi miejsce na teksturze.

P: Dlaczego `drawGeometry` jest czysto wirtualna (= 0)?

O: Bo `PrimitiveNode` to klasa abstrakcyjna - nie istnieje 'ogólny prymityw', istnieje konkretny sześcian czy sfera. `=0` wymusza, że każda podklasa musi napisać własne rysowanie. Wspólne rzeczy (material, transformacja) są w `renderSelf` w bazie - to wzorec szablonu metody.

P: Po co `glPushMatrix` / `glPopMatrix` wokół rysowania?

O: `glPushMatrix` zapisuje aktualną macierz, `glMultMatrixf` nakłada transformację obiektu, rysujemy, a `glPopMatrix` przywraca poprzedni stan. Dzięki temu transformacja jednego obiektu nie wpływa na następne.

Geometria kształtów

P: Dlaczego dla sfery normalna = pozycja wierzchołka?

O: Bo w sferze jednostkowej (promień 1) kierunek od środka do punktu jest jednocześnie kierunkiem 'na zewnątrz', czyli normalna. Wystarczy ten sam wektor użyć raz jako pozycję (po przemnożeniu przez promień) i raz jako normalną.

P: Czemu stożek rysujesz osobnymi trójkątami, a nie `TRIANGLE_FAN`?

O: W wachlarzu czubek to jeden wspólny wierzchołek z jedną normalną - cieniowanie wokół czubka wygląda wtedy płasko. Rysując osobne trójkąty, czubek każdego dostaje normalną uśrednioną z dwóch sąsiednich ścianek, więc światło płynnie przechodzi dookoła.

P: Jak liczysz nachylenie normalnej na bocznej ścianie stożka?

O: Z proporcji promienia do wysokości. $L = \sqrt{r^2 + h^2}$ to długość tworzącej. Składowa pozioma normalnej to h/L , pionowa to r/L . Pionowa jest dodatnia, bo stożek zwęża się ku górze, więc powierzchnia lekko patrzy do góry.

P: Co robi `setUVScale` w `PlaneNode`?

O: Mnoży współrzędne UV. Domyślnie 1 powtórzenie tekstury na metr - na ścianie 30m wychodzi 30 drobnych kafelków. Mniejsza skala (np. 0.2) daje większe, ładniejsze kafelki. To prosty trik bez zmiany samej geometrii.

Obstacles i kolizje

P: Jak działa wypychanie gracza z kolumny?

O: Liczę odległość gracza od osi kolumny w płaszczyźnie XZ. Jeżeli jest mniejsza niż suma promieni (kolumny + gracza), gracz jest 'w środku'. Liczę brakujący dystans i przesuwam gracza wzdłuż linii środek-gracz tak, żeby znalazł się dokładnie na brzegu.

P: Czemu przechowujesz przeszkodę dwa razy (wezeł + dane kolizji)?

O: Wezeł sceny (`CylinderNode`) służy tylko do rysowania. Do fizyki potrzebuje czystych danych (pozycja, promień, wysokość) bez całego obiektu OpenGL. Rozdzielenie 'co widac' od 'co jest fizyczne' jest czytelniejsze i szybsze.

P: Jak działają kolizje ze skrzynią (prostokątnością)?

O: Znajduje punkt na powierzchni skrzyni najbliższy graczowi (przez clamp współrzędnych do zakresu skrzyni). Jeżeli gracz jest bliżej tego punktu niż jego promień, wypycham go na zewnątrz. Gdy gracz jest w samym środku, wybieram najbliższą ścianę i wypycham w jej stronę.

P: Jak sprawdzasz, że przeszkoda zasłania cel?

O: Po znalezieniu trafionego celu mam odległość do niego ($\max T$). Sprawdzam, czy któraś przeszkoda jest przecięta przez ten sam promień przy mniejszym t . Jeżeli tak - strzał trafia w przeszkodę pierwszy, więc liczy się jako pudło.

P: Czemu w kolizjach używasz `distSq` (kwadratu odległości), a nie samej odległości?

O: Pierwiastek ($\sqrt{}$) jest wolny. Porównanie $\text{distSq} < \min R * \min R$ daje ten sam wynik co $\text{dist} < \min R$, ale bez pierwiastka. $\sqrt{}$ liczę dopiero, gdy faktycznie muszę wypchnąć gracza.